



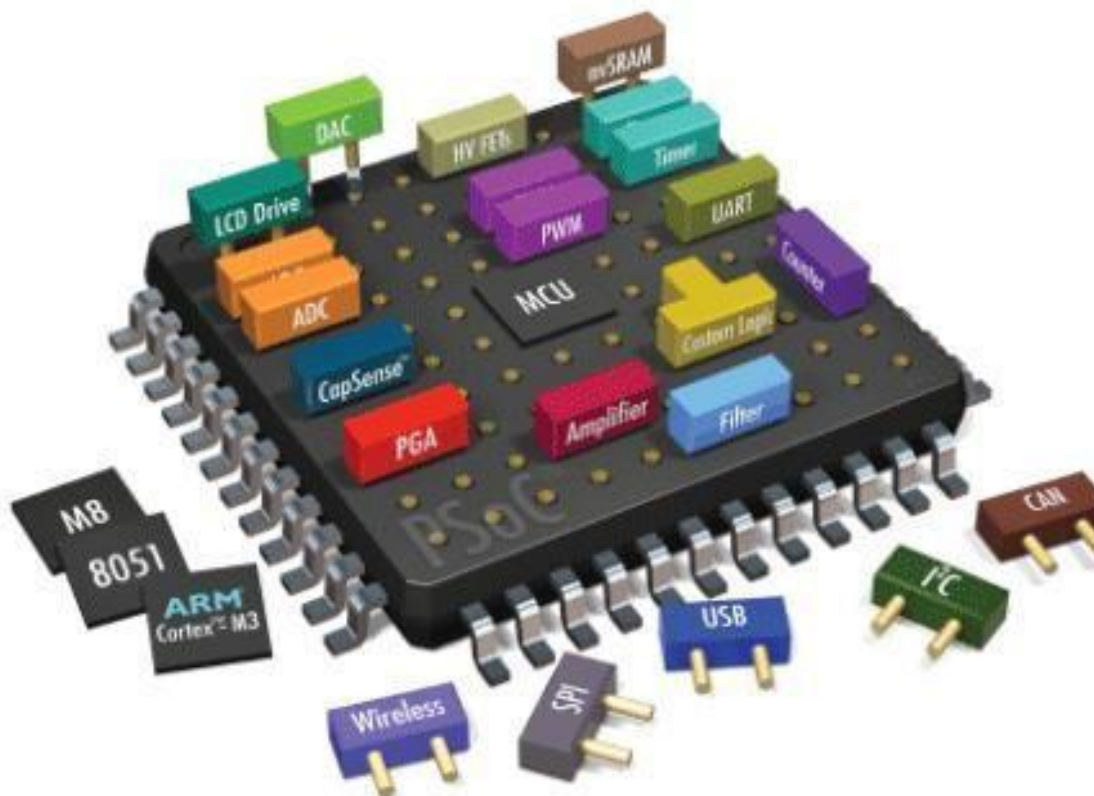
HITEC UNIVERSITY, TAXILA

DEPARTMENT OF COMPUTER ENGINEERING

Lab # 13: Understanding and practice of Timer/Counter Control of 8051.

Embedded Systems

Understanding and practice of Timer/Counter Control of 8051.



Dated:

Week 13



Objective:

This experiment demonstrates the application of 8051 Timers/Controller.

Theory:

A **timer** is a specialized type of clock which is used to measure time intervals. A timer that counts from zero upwards for measuring time elapsed is often called a **stopwatch**. It is a device that counts down from a specified time interval and used to generate a time delay, for example, an hourglass is a timer.

A **counter** is a device that stores (and sometimes displays) the number of times a particular event or process occurred, with respect to a clock signal. It is used to count the events happening outside the microcontroller. In electronics, counters can be implemented quite easily using register-type circuits such as a flip-flop.

Difference between a Timer and a Counter

The points that differentiate a timer from a counter are as follows –

Timer	Counter
The register incremented for every machine cycle.	The register is incremented considering 1 to 0 transition at its corresponding to an external input pin (T0, T1).
Maximum count rate is 1/12 of the oscillator frequency.	Maximum count rate is 1/24 of the oscillator frequency.
A timer uses the frequency of the internal clock, and generates delay.	A counter uses an external signal to count pulses.

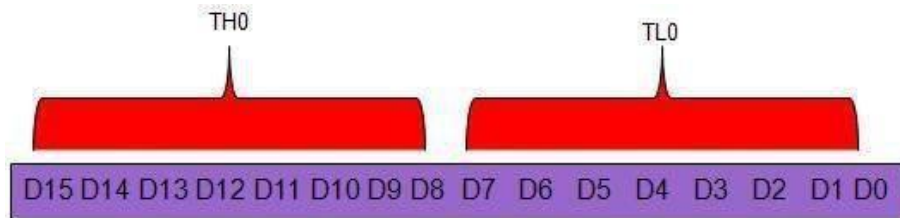
Timers of 8051 and their Associated Registers

The 8051 has two Timers/Counters, which is Timer0/Counter0 and Timer1/Counter1.. They can be used as timers or as event counters. Both Timer 0 and Timer 1 are 16-bit wide. Since the 8051 follows an 8-bit architecture, each 16 bit is accessed as two separate registers of low-byte and high-byte.



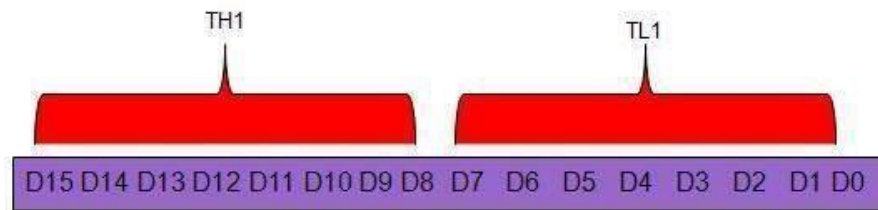
Timer 0 Register

The 16-bit register of Timer 0 is accessed as low- and high-byte. The low-byte register is called TL0 (Timer 0 low byte) and the high-byte register is called TH0 (Timer 0 high byte). These registers can be accessed like any other register. For example, the instruction **MOV TL0, #4H** moves the value into the low-byte of Timer #0.



Timer 1 Register

The 16-bit register of Timer 1 is accessed as low- and high-byte. The low-byte register is called TL1 (Timer 1 low byte) and the high-byte register is called TH1 (Timer 1 high byte). These registers can be accessed like any other register. For example, the instruction **MOV TL1, #4H** moves the value into the low-byte of Timer 1.



8-bit COUNTER OPERATION

First let us consider a simple 8-bit counter. Since this is a modulo-8 set up we are concerned with 256 numbers in the range 0 to 255 ($2^8 = 256$). The counter will count in a continuous sequence as follows:

Hex	Decimal
00h	0
01h	1
02h	2
.	.
FEh	254
FFh	255
00h	0
01h	1
02h	2

————> here the counter **overflows** to zero



HITEC UNIVERSITY, TAXILA

DEPARTMENT OF COMPUTER ENGINEERING

Lab # 13: Understanding and practice of Timer/Counter Control of 8051.



Supposing we were to initialize this Timer/Counter with a number, say 252, then the counter would overflow after just four event pulses, i.e:

FCh	252	
FDh	253	
FEh	254	
FFh	255	
00h	0	————> here the counter overflows to
zero 01h	1	
02h	2	

An 8-bit counter can count 255 events before overflow, and overflows on the 256th event. When initialized with a predefined value of say 252 it overflows after counting just four events. Thus the number of events to be counted can be programmed by pre-loading the counter with a given number value.

8-bit TIMER OPERATION

The 8051 internally divides the processor clock by 12. If a 12 MHz processor clock is used then a 1 MHz instruction rate clock, or a pulse once every microsecond, is realised internally within the chip. If this 1 microsecond pulse is connected to a Timer/Counter input, in place of an *event* input, then the Timer/Counter becomes a timer which can delay by up to 255 microseconds. There is a clear difference between a timer and a counter. The counter will count events, up to 255 events before overflow, and the timer will count time pulses, thus creating delays up to 255 microseconds in our example.

To be precise we would refer to the *counter* as an *event counter* and we would refer to the *timer* as an *interval timer*.

If the timer is initialized to zero it will count 256 microseconds before overflow. If the timer is initialized to a value of 252, for example, it will count just 4 microseconds before overflow. Thus this timer is programmable between 1 microsecond and 256 microseconds.

Before programming we should know the Timer Registers. 8051 have 5 registers for timer operation.

Registers used

- TMOD (Timer mode Register)
- TCON (Timer control Register)
- THx (Timer High Register) (x means weather 0 or 1. It indicates timer 0 or Timer 2)
- TLx (Timer Low Register) (x means weather 0 or 1. It indicates timer 0 or Timer 2)

TMOD Register

Gate	C/T	M1	M0	Gate	C/T	M1	M0
Timer 1				Timer 0			



Gate: This bit indicates How to control the timer weather by software, or hardware. If we set this by 0 we can start and stop the timer by software(coding). If we set 1 timer is controlled by hardware. we have the dedicated pin in Port 0.

C/T: This bit used to select timer or counter. 0 for Timer and 1 for counter.

M0, M1: This bit are used to select the timer modes like below.

M1	M0	Mode
0	0	Mode 0: 13-bit mode (seldom used).
0	1	Mode 1: 16-bit mode
1	0	Mode 2: 8-bit mode (with auto reload feature)
1	1	Mode 3: 8-bit split timer

Bit	Name	Explanation of Function	Timer
7	GATE1	When this bit is set the timer will only run when INT1 (P3.3) is high. When this bit is clear the timer will run regardless of the state of INT1.	1
6	C/T1	When this bit is set the timer will count events on T1 (P3.5). When this bit is clear the timer will be incremented every machine cycle.	1
5	T1M1	Timer mode bit (see below)	1
4	T1M0	Timer mode bit (see below)	1
3	GATE0	When this bit is set the timer will only run when INT0 (P3.2) is high. When this bit is clear the timer will run regardless of the state of INT0.	0
2	C/T0	When this bit is set the timer will count events on T0 (P3.4). When this bit is clear the timer will be incremented every machine cycle.	0
1	T0M1	Timer mode bit (see below)	0
0	T0M0	Timer mode bit (see below)	0



13-bit Time Mode (Mode 0)

Timer mode “0” is a 13-bit timer. This is a relic that was kept around in the 8051 to maintain compatibility with its predecessor, the 8048. Generally the 13-bit timer mode is not used in new development.

When the timer is in 13-bit mode, TLx will count from 0 to 31. When TLx is incremented from 31, it will “reset” to 0 and increment THx. Thus, effectively, only 13 bits of the two timer bytes are being used: bits 0-4 of TLx and bits 0-7 of THx. This also means, in essence, the timer can only contain 8192 values. If you set a 13-bit timer to 0, it will overflow back to zero 8192 machine cycles later.

Again, there is very little reason to use this mode and it is only mentioned so you won’t be surprised if you ever end up analyzing archaic code which has been passed down through the generations (a generation in a programming shop is often on the order of about 3 or 4 months).

16-bit Time Mode (Mode 1)

Timer mode “1” is a 16-bit timer. This is a very commonly used mode. It functions just like 13-bit mode except that all 16 bits are used.

TLx is incremented from 0 to 255. When TLx is incremented from 255, it resets to 0 and causes THx to be incremented by 1. Since this is a full 16-bit timer, the timer may contain up to 65536 distinct values. If you set a 16-bit timer to 0, it will overflow back to 0 after 65,536 machine cycles.

8-bit Time Mode (Mode 2)

Timer mode “2” is an 8-bit auto-reload mode. What is that, you may ask? Simple. When a timer is in mode 2, THx holds the “reload value” and TLx is the timer itself. Thus, TLx starts counting up. When TLx reaches 255 and is subsequently incremented, instead of resetting to 0 (as in the case of modes 0 and 1), it will be reset to the value stored in THx.

For example, let’s say TH0 holds the value FDh and TL0 holds the value FEh. If we were to watch the values of TH0 and TL0 for a few machine cycles this is what we see:

Machine Cycle	TH0 Value	TL0 Value
1	FDh	FEh
2	FDh	FFh
3	FDh	FDh
4	FDh	FEh
5	FDh	FFh



6	FDh	FDh
7	FDh	FEh

As you can see, the value of TH0 never changed. In fact, when you use mode 2 you almost always set THx to a known value and TLx is the SFR that is constantly incremented.

What is the benefit of auto-reload mode? Perhaps you want the timer to always have a value from 200 to 255. If you use mode 0 or 1, you'd have to check in code to see if the timer had overflowed and, if so, reset the timer to 200. This takes precious instructions of execution time to check the value and/or to reload it. When you use mode 2 the microcontroller takes care of this for you. Once you've configured a timer in mode 2 you don't have to worry about checking to see if the timer has overflowed nor do you have to worry about resetting the value—the microcontroller hardware will do it all for you.

Split Timer Mode (Mode 3)

Timer mode “3” is a split-timer mode. When Timer 0 is placed in mode 3, it essentially becomes two separate 8-bit timers. That is to say, Timer 0 is TL0 and Timer 1 is TH0. Both timers count from 0 to 255 and overflow back to 0. All the bits that are related to Timer 1 will now be tied to TH0.

While Timer 0 is in split mode, the real Timer 1 (i.e. TH1 and TL1) can be put into modes 0, 1 or 2 normally—however, you may not start or stop the real timer 1 since the bits that do that are now linked to TH0. The real timer 1, in this case, will be incremented every machine cycle no matter what.

The only real use I can see of using split timer mode is if you need to have two separate timers and, additionally, a baud rate generator. In such case you can use the real Timer 1 as a baud rate generator and use TH0/TL0 as two separate timers.

TCON Register

Finally, there is one more SFR that controls the two timers and provides valuable information about them. The TCON SFR has the following structure:

**TCON : Timer/Counter Control Register (Bit Addressable)**

TF1	TR1	TF0	TR0	IE1	IT1	IE0	IT0
-----	-----	-----	-----	-----	-----	-----	-----

TF1	TCON.7	Timer 1 overflow flag. Set by hardware when the Timer/Counter 1 overflows. Cleared by hardware as processor vectors to the interrupt service routine.
TR1	TCON.6	Timer 1 run control bit. Set/cleared by software to turn Timer/Counter ON/OFF.
TF0	TCON.5	Timer 0 overflow flag. Set by hardware when the Timer/Counter 0 overflows. Cleared by hardware as processor vectors to the service routine.
TR0	TCON.4	Timer 0 run control bit. Set/cleared by software to turn Timer/Counter 0 ON/OFF.
IE1	TCON.3	External Interrupt 1 edge flag. Set by hardware when External interrupt edge is detected. Cleared by hardware when interrupt is processed.
IT1	TCON.2	Interrupt 1 type control bit. Set/cleared by software to specify falling edge/low level triggered External Interrupt.
IE0	TCON.1	External Interrupt 0 edge flag. Set by hardware when External Interrupt edge detected. Cleared by hardware when interrupt is processed.
IT0	TCON.0	Interrupt 0 type control bit. Set/cleared by software to specify falling edge/low level triggered External Interrupt.

As you may notice, we have only 4 timer defined of the 8 bits. That is because the other 4 bits of the SFR don't have anything to do with timers, they have to do with Interrupts and they will be discussed in the chapter that addresses interrupts.

A new piece of information in this chart is the column "bit address." This is because this SFR is "bit-addressable." What does this mean? It means if you want to set the bit TR1—which is the sixth bit of TCON—you can write in the program as

TR1=1; as of in the case of byte address you have to write as

TCON=0x40; this is not needed in this SFR

This has the benefit of setting the high bit of TCON without changing the value of any of the other bits of the SFR. Usually when you start or stop a timer you don't want to modify the other values in TCON, so you take advantage of the fact that the SFR is bit-addressable.

THx Register & TLx Register

In these register, the counting values are stored.

Programming of the Timer/Counter

Let's look at how to do the following:

- Configure the Timer/Counter as a TIMER or as a COUNTER
- Program the Timer/Counter with a value between 0 and 255
- Enable and disable the Timer/Counter
- How to know when the timer has overflowed – interrupt vs. polling.

Initializing a Timer

Now that we've discussed the timer-related SFRs we are ready to write code that will initialize the timer and start it running. In our examples we will discussed about only Timer 0. But Timer 1



HITEC UNIVERSITY, TAXILA

DEPARTMENT OF COMPUTER ENGINEERING

Lab # 13: Understanding and practice of Timer/Counter Control of 8051.

also operates same like Timer 1.



As you'll recall, we first must decide what mode we want the timer to be in. In this case we want a 16-bit timer that runs continuously; that is to say, it is not dependent on any external pins.

We must first initialize the TMOD SFR. Since we are working with timer 0 we will be using the lowest 4 bits of TMOD. The first two bits, GATE0 and C/T0 are both 0 since we want the timer to be independent of the external pins. 16-bit mode is timer mode 1 so we must clear T0M1 and set T0M0. Effectively, the only bit we want to turn on is bit 0 of TMOD.

TMOD=0x01;

Timer 0 is now in 16-bit timer mode. However, the timer is not running. To start the timer running we must set the TR0 bit we can do that by executing the instruction.

TR0=1;

Upon executing these two instructions timer 0 will immediately begin counting, being incremented once every machine cycle (every 12 crystal pulses).

Then the timer values have to be loaded in THx and TLx registers so that the timer runs from the mentioned value say,

TH0=0xEE;

TL0=0x00; this is for the 5ms

This can be calculated by using the formula,

General formula for timer

(65536-xx)*machine cycle= Desired time

Where xx is a 16 bit no having value of TH0 and TL0.

When timer run in 16 bit mode with 11.0592 MHz max delay.

That can be generated is 72 ms almost. Now let say u use 11.0592MHz crystal then Machine cycle is calculated as for every crystal value

$T=1/F$ (for one clock pulse)

$T=12/F$ (for twelve clock pulse= 1 machine cycle)

$T=1.085\mu s$.

Example

$(65536-xx)*1.085\mu s= 5$

ms $(65536-xx)=4608$

$xx=60928$

$xx=0xEE00$. (This is Hex value)

Detecting Timer Overflow

Often it is necessary to just know that the timer has reset to 0. That is to say, you are not particularly interest in the value of the timer but rather you are interested in knowing when the timer has overflowed back to 0.

Whenever a timer *overflows* from it is highest value back to 0, the microcontroller automatically sets the TFx bit in the TCON register. This is useful since rather than checking the exact value of the timer you can just check if the TFx bit is set. If TF0 is set it means that timer 0 has overflowed; if TF1 is set it means that timer 1 has overflowed. We can use this approach to cause the program to execute a fixed delay. say

while (TF0==0); for timer 0 overflow



Then the TF0 flag has to be manually set to 0 for the recursive purpose of the timer.

The for loop has to be included in order to call the desired time delay for a no times so that the loop can achieve the desired time delay in the range of seconds.

Say in the below example the 5ms program is executed repetitively for 200 times so that the one second delay is achieved.

The programming of 8051 Timers can be done by using either polling method or by using interrupt. In polling, the microcontroller keeps monitoring the status of Timer flag. While doing so, it does no other operation and consumes all its processing time in checking the Timer flag until it is **raised on a rollover**. In interrupt method controller responds to only when the Timer flag is raised. The interrupt method prevents the wastage of controller's processing time unlike polling method.

Polling is mostly used for time delay generation and interrupt method is more useful when waveforms are to be generated or some action has to be repeated in fixed delays.

(i) Polling Method

The polling method involves the following algorithm:

1. Configure the Timer mode by passing a hex value into the TMOD register. This will tell the controller about which Timer is be used; the mode of Timer; operation (to be used as timer or counter); and whether external interrupt is required to start Timer.
2. Load the initial values in the Timer low TLx and high THx byte. (x = 0/1)
3. Start the Timer by setting TRx bit.
4. Wait while the Timer flag TFx is raised.
5. Clear the Timer flag. The Timer flag is raised when Timer **rolls over** from FFFFH to 0000H. If the Timer is not stopped, it will start updating from 0000H in case of modes 0 & 1 while with initial value in case of mode 2. If TFx is not cleared, controller will not be able to detect next rollover.
6. Stop the Timer by clearing TRx bit. If TRx bit is not cleared the Timer will restart updating from 0000H after the rollover in case of modes 0 and 1 while with initial value in case of mode 2.

(ii) Interrupt Method

The interrupt method makes use of a register called **Interrupt Enable (IE)** register. An 8051 microcontroller has 6 hardware interrupts. The interrupts refer to a notification, communicated to the controller, by a hardware device or software, on receipt of which controller skips temporarily whatsoever it was doing and responds to the interrupt.



The controller starts the execution of an **Interrupt Service Routine (ISR)** or Interrupt Handler which is a piece of code that tells the processor or controller **what to do on receipt of an interrupt**. After the execution of ISR, controller returns to whatever it was doing earlier (before the interrupt was received).

The Interrupt Enable register has the following bits which enable or disable the hardware interrupts of 8051 microcontroller.

IE :	EA	-	ET2	ES	ET1	EX1	ET0	EX0
-------------	-----------	----------	------------	-----------	------------	------------	------------	------------

Bit Values of IE Register of 8051 Microcontroller

When EA (IE.7) is set (=1), interrupts are enabled. When clear (EA=0), interrupts are disabled and controller does not respond to any interrupts.

ET0, ET1 & ET2 (IE.3, IE.4 & IE.5) are Timer interrupt bits. When set (high) the timer are enabled and when cleared (low) they are disabled. (8052 controllers have three Timers, so ET2 is its Timer 2 interrupt bit.) The ISR corresponding to these interrupts are executed when the TFX flags of respective Timers are raised. For more details on other IE register bits, refer [Interrupts with 8051](#).

Note that IE register is bit addressable.

Timer programming using Timer interrupts involves following algorithm.

1. Configure the Timer mode by passing a hex value to TMOD register.
2. Load the initial values in the Timer low TLx and high THx byte.
3. Enable the Timer interrupt by passing hex value to IE register or setting required bits of IE register.
4. Start the Timer by setting TRx bit.
5. Write Interrupt Service Routine (ISR) for the Timer interrupt. For example,
6. If the Timer has to be stopped after once the interrupt has occurred, the ISR must contain the statement to stop the Timer.
7. If a routine written for Timer interrupt has to be repeated again and again, the Timer run bit need not be cleared. But it should be kept in mind that **Timer will start updating from 0000H and not the initial values in case of mode 0 and 1**. So the initial values must be reloaded in the interrupt service routine.



Programming the timers in the 8051 microcontroller

Example 1:

Count the number of pulses from the 555 oscillator output and show the result on 7-seg displays DS3 and DS4. Connect pins 2 and 3 of JP4 together using a jumper cap.

The Output of the 555 oscillator is connected to the T0 (P3.4) pin. If a 1-to-0 transition on P3.4 is detected, the A register will be incremented by one and the new count value is then written to port 2 to drive DS3 and DS4.

Program:

ORG 000H

NEXT:

```
M
O
V
A
,
#
0
M
O
V
P
2
,
#
0

JB    P3.4,$

A
D
D

A
,
#
1
D
A

A
M
```



HITEC UNIVERSITY, TAXILA

DEPARTMENT OF COMPUTER ENGINEERING

Lab # 13: Understanding and practice of Timer/Counter Control of 8051.

O	N
V	
P	E
2	
,	X
A	
	T
JNB P3.4,\$	
	E
J	
	N
M	
	D
P	

For speaker:

If MOV C,P3.4 and MOV P3.7,C instructions are added to the program above, then it output the pulse to the speaker.

ORG 000H

NEXT:

M
O
V
A
,
#
0
M
O
V
P
2
,
#
0
JB P3.4,\$
M
O
V
C



HITEC UNIVERSITY, TAXILA

DEPARTMENT OF COMPUTER ENGINEERING

Lab # 13: Understanding and practice of Timer/Counter Control of 8051.

,
P
3
.
4
M
O
V
P
3
.
7
,
C

ADD A,#1



```
DA    A
MOV P2,A

JNB   P3.4,$

MOV C,P3.4
MOV P3.7,C

JMP   NEXT

END
```

Example 2:

Count the number of pulses from the photo interrupter PH1 and show the result on 7-seg displays DS3 and DS4. Connect pins 1 (T1) and 2 (PH1) of JP3 together using a jumper cap.

Program:

```
ORG 000H

NEXT:

MOV A,0
MOV P2,A

JNB P3.4

MOV A,P3
MOV P2,A
```



HITEC UNIVERSITY, TAXILA

DEPARTMENT OF COMPUTER ENGINEERING

Lab # 13: Understanding and practice of Timer/Counter Control of 8051.

,	3
\$	.
A	5
D	,
D	\$
A	J
,	
#	M
1	
D	P
A	
A	
M	N
O	
V	E
P	
2	X
,	
A	T
J	
N	E
B	
	N
P	
	D

Example 3:

Use timer/counter 1 to count the output pulses of PH1 and show the result on 7-seg displays DS3 and DS4. Connect pins 1 (T1) and 2 (PH1) of JP3 together using a jumper cap.

Program:

```
ORG 000H

MOV P2,#0
MOV TMOD,#01010000B
MOV TH1,#0
```



HITEC UNIVERSITY, TAXILA

DEPARTMENT OF COMPUTER ENGINEERING

Lab # 13: Understanding and practice of Timer/Counter Control of 8051.

NEXT:

M
O
V
T
L
1
,

0
S
E
T
B
T
R
1

M
O
V
R
0
,
T
L
1
C
A
L
L
B
I
N
T
O
B
C
D

JMP NEXT

=====



HITEC UNIVERSITY, TAXILA

DEPARTMENT OF COMPUTER ENGINEERING

Lab # 13: Understanding and practice of Timer/Counter Control of 8051.

; BINARY TO BCD CONVERT

=====

BINTOBCD:

CLR C

MOV R1,#0

MOV R2,#8

NBIT:

M

O

V

A

,

R

0

R

L

C

A

M

O

V

R

0

,

A

M

O

V

A

,

R

1

A

D

D

C

A

,

R

1

D

A

A

M



HITEC UNIVERSITY, TAXILA

DEPARTMENT OF COMPUTER ENGINEERING

Lab # 13: Understanding and practice of Timer/Counter Control of 8051.

O	M
V	
R	O
1	
,	V
A	
D	P
J	2
N	,
Z	R
R	1
2	R
,	E
N	T
B	END
I	
T	

Example 4:

Program **timer/counter 0** operating in **mode 0** and blink LEDs (D5) alternately. The ON or OFF time is 1 second. Place SW2_4 in ON position.

Program:

```
ORG 000H
JMP MAIN
ORG 00BH
JMP
TIMER0

ORG 100H
```



HITEC UNIVERSITY, TAXILA

DEPARTMENT OF COMPUTER ENGINEERING

Lab # 13: Understanding and practice of Timer/Counter Control of 8051.

MAIN:

M
O
V
T
M
O
D
,

0
0
0
0
0
0
0
0
0
0
B
M
O
V
I
E
,

1
0
0
0
0
0
1
0
B
M
O
V
T
H
0
,
#



HITEC UNIVERSITY, TAXILA

DEPARTMENT OF COMPUTER ENGINEERING

Lab # 13: Understanding and practice of Timer/Counter Control of 8051.

```
8                                     P
1                                     2
MOV TL0,#92                          ,
                                     #
M                                     0
O                                     F
V                                     F
R                                     H
0                                     S
,                                     E
#                                     T
2                                     B
0                                     T
0                                     R
M                                     0
O
V                                     JMP $
```

```
=====
;
TIMER0:
    CLR  TR0
    DJNZ R0,BACK
    XRL  P2,#0FFH
    MOV R0,#200
BACK:
    MOV TH0,#50
    MOV TL0,#00

    SETB TR0
    RETI

    END
```

Example 5:

Program **timer/counter 0** operating in **mode 1** and sequentially turn on/off LEDs from the leftmost to the rightmost. The ON or OFF time is 0.5 second. Place SW2_4 in ON position.

Program:

```
    ORG 000H
    JMP MAIN
    ORG 00BH
    JMP
    TIMER0

                                ORG 100H

MAIN:
```



HITEC UNIVERSITY, TAXILA

DEPARTMENT OF COMPUTER ENGINEERING

Lab # 13: Understanding and practice of Timer/Counter Control of 8051.

M
O
V
T
M
O
D
,

0
0
0
0
0
0
0
0
0
1
B
M
O
V
I
E
,

1
0
0
0
0
0
1
0
B
MOV TH0,#>(65536-50000)



```
MOV TL0,#<(65536-50000)
```

```
MOV R0,#10
```

```
MOV A,#10000000B
```

```
SETB TR0
```

```
JMP $
```

```
=====
```

```
TIMER0:
```

```
CLR TR0
```

```
DJNZ R0,BACK
```

```
MOV P2,A
```

```
RR A
```

```
MOV R0,#10
```

```
BACK:
```

```
MOV TH0,#>(65536-50000)
```

```
MOV TL0,#<(65536-50000)
```

```
SETB TR0
```

```
RETI
```

```
END
```

Example 6:

Program **timer/counter 0** operating in **mode 2** and sequentially turn on/off LEDs from the rightmost to the leftmost. The ON or OFF time is 0.5 second. Place SW2_4 in ON position.

Program:

```
ORG 000H
```

```
JMP MAIN
```

```
ORG 00BH
```

```
JMP
```

```
TIMER0
```

```
ORG 100H
```

```
MAIN:
```

```
M
```

```
O
```

```
V
```

```
T
```

```
M
```



HITEC UNIVERSITY, TAXILA

DEPARTMENT OF COMPUTER ENGINEERING

Lab # 13: Understanding and practice of Timer/Counter Control of 8051.

O
D
,

0
0
0
0
0
0
0
1
0
B
M
O
V
I
E
,

1
0
0
0
0
0
0
1
0
B
M
O
V
T
H
0
,

(
2
5
6
-
2
5
0

)
M
O
V
T
L
0
,

(
2
5
6
-
2
5
0
)



```
MOV A,#10000000B
MOV R0,#10
MOV R1,#200
```

```
SETB TR0
```

```
JMP $
```

```
;=====
;
TIMER0:
    DJNZ R1,BACK
    MOV R1,#200
    DJNZ R0,BACK
    MOV P2,A
    RR    A
    MOV R0,#10
BACK:
    RETI

    END
```

Example 7:

Program **timer/counter 0** operating in **mode 3** to extend the delay. Blink two set of LEDs alternately The ON or OFF time is 1 second. Place SW2_4 in ON position.

Program:

```
    ORG 000H
    JMP MAIN
    ORG 00BH
    JMP
TIMER0 ORG
        01BH
    JMP
TIMER1

                                ORG 100H

MAIN:                                M
                                    O
                                    V
                                    T
                                    M
                                    O
```



HITEC UNIVERSITY, TAXILA

DEPARTMENT OF COMPUTER ENGINEERING

Lab # 13: Understanding and practice of Timer/Counter Control of 8051.

D	M
,	O
#	V
0	T
0	L
0	0
0	,
0	#
0	(
1	2
1	5
B	6
M	-
O	2
V	5
I	0
E	)
,	
#	M
1	O
0	V
0	A
0	,
1	#
0	0
1	0
0	0
B	0
M	1
O	1
V	1
T	1
H	B
0	M
,	O
#	V
(	P
2	2
5	,
6	A
-	
2	
5	
0	
)	



```
MOV R0,#10
MOV R1,#200
```

```
SETB TR0
```

```
JMP $
```

```
=====
;
TIMER0:
    CLR  TR0
    MOV TL0,#(256-250)
    SETB TR1
    RETI
```

```
=====
;
TIMER1:
    CLR  TR1
    MOV TH0,#(256-250)
    DJNZ R1,BACK
    MOV R1,#200
    DJNZ R0,BACK
    SWAP A
    MOV P2,A
    MOV R0,#10
BACK:
    SETB TR0
    RETI

    END
```

Lab Tasks:

1. Display the examples in the manual on MTS-51 and analyze the output.
2. Write a program by using timer/counter 0 to generate a 1 KHz square wave on P2.0.
View the output waveform using as oscilloscope.

Lab Performance Evaluation:

Read and practice the manual. Performance is evaluated at the end of lab based on successfully running and understanding of examples, tasks and viva using defined rubrics.

Lab Report Evaluation:

Submit the Lab Report by writing all the examples and tasks which must include the following:



HITEC UNIVERSITY, TAXILA

DEPARTMENT OF COMPUTER ENGINEERING

Lab # 13: Understanding and practice of Timer/Counter Control of 8051.

1. Brief description about how its work (3-5 lines)
2. The code
3. The results (Screenshot)

Important note: Lab Report must be according to the Lab Report Format available on Google Classroom and must be submitted on time. Two similar reports will get zero marks. So, don't try to copy your fellow reports. Lab report must be submitted in group of three maximum.